Vuex

状态：就是Vue组件中data里面的属性，我们也将叫做数据。

管理：就是对于项目中数据的操作管理。

管理什么状态呢？

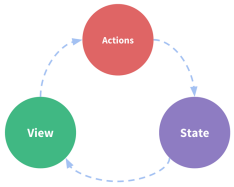
用户的登录状态、地理位置信息、商品的收藏、购物车的物品。

单界面状态管理

data选项中的属性就是我们的状态state

模板中使用{{ }}Mastach语法渲染数据就是我们的视图view

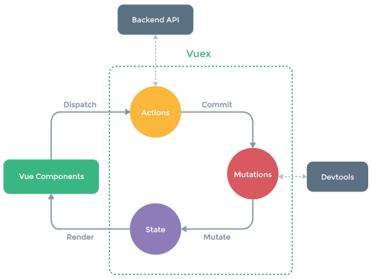
methods选项中定义的修改数据的方法就是actions



多界面状态管理

Vuex是将不同组件的共享状态抽离出来,以全局单例模式进行管理.

这样不管在哪个组件中都可以获取状态或触发修改状态的行为.



vuex的基本使用

1.提取出一个公共的store对象，用于保存在多个组件中共享的状态

2.将store对象放置在new Vue对象中，这样可以保证在所有的组件中都可以使用到

3.在其他组件中使用store对象中保存的状态即可

通过this.$store.state.属性的方式来访问状态

通过this.$store.commit('mutation中方法')来修改状态

1.安装Vuex　npm i vuex -S
2.在store下的index.js引入Vuex
import Vue from 'vue'
import Vuex from 'vuex'

```
Vue.use(Vuex)//安装
const store =new Vuex.Store({
    state:{},
    mutations:{},
    actions:{},
    getters:{},
    modules:{}
})
```

3.在main.js中引入并注入

```
import store from './store'
new Vue({
  // 注入vuex
  store,
  render: h => h(App),
}).$mount('#app')

//state和mutations已经定义好了状态和方法
//在state中自定义共享状态
state:{//存储状态的
        // 自定义共享状态
        count:0,
        stu:[
            {id:1001,name:"123"},
            {id:1003,name:"福鼠"},
            {id:1004,name:"追梦"},
        ],
        user:{
            name:'随便',
            sex:"随机"
        }
    },
// Vuex中的Store状态（state）更新的唯一方式：提交Mutation
mutations:{ //定义mutations
        //第一个参数就是state
        // this是store对象
        // state(store中的state),(Payload:提交方法携带的参数)
        addTen(state,obj){ //+10
            state.count +=obj.ten
        },
        // 向stu数组中添加一条数据
        addStu(state,payload){
            state.stu.push(payload) //向数组中添加一条数据
        },
        updUser(state,age){ //给user对象添加新属性age
            // state.user.age="18";
            // state.user = { ...state.user, "age": 12 }
```

```
            Vue.set(state.user, 'age', 123)
        }
    }

<h1>这个是myvuex组件</h1>
<!-- 直接操作state里面的数据count是不会有响应式的 --
<button@click="$store.state.count++">+</button>
<button@click="$store.state.count--">-</button>

<button@click="addTen">mutations +10</button>
<button @click="addStu">添加学员</button>
//注意事项:
//我们通过提交mutation的方式，而非直接改变store.state.count。
//这是因为Vuex可以更明确的追踪状态的变化，所以不要直接改变store.state.count的值。
data(){
      return{
          stu:{id:1005,name:'测试'}
      }
  },
methods:{
    addTen(){// 触发mutations函数，指定额外参数
      this.$store.commit("addTen",10)//字符串形式提交
    },
    addStu(){// 触发mutations函数，指定额外参数
       this.$store.commit("addStu",this.stu) //字符串形式提交
    },
     addTen(){
      this.$store.commit({ //对象提交方式
           type:"addTen",
           two:10
      })
    },
    updUser(){ //给mutations中的stu对象添加新属性
         this.$store.commit({ //对象提交方式
         type:"updUser"})
      },
}
```

以上我们记录了state和mutations的基本用法

下面我们接受看actions和getters

1.在index.js中自定义状态

```
export default new Vuex.Store({
  state: {
      count:0,
      info:[
          {id:1,name:'开发者1',age:20},
```

```
        {id:2,name:'开发者2',age:26},
        {id:3,name:'开发者3',age:28},
        {id:4,name:'开发者4',age:18},
        {id:5,name:'开发者5',age:12},
        {id:6,name:'开发者6',age:32}
    ]
  },
  mutations: {
    // state,payload
    increment(state,payload){
        // 如果mutation是一个异步函数，那么devtools不能跟踪数据的变化
        // mutaiton负责同步修改状态
         state.count+=payload
    }
  },
  actions: {
    asyncIcrement(context,payload){
        // 上下文对象
        // console.log(context)
        setTimeout(()=>{//重点
              context.commit("increment",payload)
        },2000)
    }
  },
  getters:{ //store的计算属性
   // 计算count的平方
    quadratic(state){
      // 第一个参数，状态state
      return state.count*state.count
    },
    // 筛选数据
    filterInfo(state,getters){
         console.log(getters)
         // age返回的不是一个不确定的值，而是一个函数
          return (age)=>{ // 传递参数
              console.log(age)
              return state.info.filter(res => res.age> age)
          }
    },
    testGettes(state,getters){
        console.log(getters)
      return  getters.quadratic
    }
  }
})
```

2.在组件中去使用

```
<h2>这是MyVuex组件</h2>
```

```
<button @click="add">+1</button>
<button @click="asyAdd">异步+1</button>
<h3>平方：{{$store.getters.quadratic}}</h3>
<h1>testGettes: {{$store.getters.testGettes}}</h1>
<li v-for="itme in info" >{{itme}}</li>//迭代输出年龄大于18的开发者信息（信息在数组中）
        methods:{
            add(){
                // 触发mutations中的函数
                this.$store.commit('increment',5)
            },
            asyAdd(){
                // 触发actions函数，并且传参
                //注意：异步使用dispatch
                this.$store.dispatch("asyncIcrement",10)
            }
        },
        data(){
            return{//触发getters函数，并且进行筛选
                info:this.$store.getters.filterInfo(18)
            }
        }
```

modules

为什么在Vuex中我们要使用模块呢？

Vue使用单一状态树,那么也意味着很多状态都会交给Vuex来管理。

当应用变得非常复杂时,store对象就有可能变得相当臃肿。

为了解决这个问题，Vuex允许我们将store分割成模块(Module)，而每个模块拥有自己的state、mutation、action、getters等

```
import myUser from './myuser.js'//这是外部引入的状态
import cat from './cat.js'

const myCount={}

export default new Vuex.Store({
  state:{
      num:2
  },
  modules: { // 模块选项
     // 引用myuser模块
      u:myUser,//这是外部引入的状态
      c:myCount, //这是默认的状态树
```

```
    cat//这是外部引入的状态（同名可缩写）
  }
})
```